



AI-Powered RAG-Based Legal Assistant

Project Explanation Document — B.Tech Major Project

Reference implementation studied: github.com/lawglance/lawglance
Department of CS&E, Jaypee University of Engineering & Technology, Guna

1. Project Overview

LawGlance is a free, open-source, AI-powered legal assistant that uses Retrieval-Augmented Generation (RAG) to answer legal questions in plain language. Instead of asking a user to manually search through dense legal texts — bare acts, sections, and case-specific procedures — the system retrieves the exact relevant legal text and uses a large language model (LLM) to turn it into a clear, conversational answer, grounded strictly in that retrieved text.

The system currently covers Indian law, including:

- The Indian Constitution
- The Bharatiya Nyaya Sanhita, 2023 (replacing the IPC)
- The Bharatiya Nagarik Suraksha Sanhita, 2023 (replacing the CrPC)
- The Bharatiya Sakshya Adhiniyam, 2023 (replacing the Evidence Act)
- The Consumer Protection Act, 2019
- The Motor Vehicles Act, 1988
- The Information Technology Act, 2000
- The POCSO Act, 2012
- The Sexual Harassment of Women at Workplace Act, 2013

It is delivered as a conversational, chat-style web application with persistent session memory, so users can ask natural follow-up questions the same way they would with a human legal aide.

2. Problem Statement

2.1 The Problem

Access to legal knowledge in India is heavily gated by cost, language, and complexity:

- **Legal literacy gap:** Most citizens cannot interpret statutory language (e.g. sections of the BNS or Constitution) without professional help.
- **Cost of consultation:** A basic legal opinion from a lawyer is often unaffordable or inaccessible for routine questions (tenancy disputes, consumer complaints, minor traffic violations, etc.).
- **Fragmented sources:** Relevant law is spread across multiple acts and amendments (e.g. the 2023 criminal law overhaul replacing IPC/CrPC/Evidence Act), making manual lookup slow and error-prone.
- **Generic AI chatbots hallucinate:** A general-purpose LLM asked a legal question will often generate plausible-sounding but legally incorrect or outdated answers, because it is not grounded in the actual statutory text.

2.2 Why This Needs RAG (Not Just an LLM)

A plain LLM answering legal questions from parametric memory is risky — training data may be outdated, incomplete, or simply wrong on jurisdiction-specific detail. RAG addresses this by forcing the model to answer only from text retrieved at query time from a verified legal knowledge base, which:

- Grounds every answer in an actual statutory passage, reducing hallucination

- Allows the knowledge base to be updated (e.g. new acts, amendments) without retraining the model
- Makes answers auditable — the retrieved source passage can, in principle, be traced back to the exact act and section

3. Objectives

1. Build a RAG pipeline that retrieves the most relevant statutory passages for a user's natural-language legal question.
2. Ground LLM responses strictly in retrieved context to minimize hallucination on legal facts.
3. Support multi-turn conversations, where follow-up questions are understood in the context of chat history.
4. Provide a simple, accessible chat UI usable by non-technical laypersons.
5. Cache repeated queries and responses to reduce latency and API cost.
6. Design the system to be extensible to more acts, more jurisdictions, and more languages over time.

4. Data

4.1 Data Sources

The knowledge base is built from the full text of the statutes listed in Section 1 — the Constitution of India and the acts governing criminal law, consumer protection, motor vehicles, IT law, and offences against women and children. These are official, publicly available legal texts (bare acts), not summaries or third-party commentary, which keeps the retrieval grounded in primary legal sources.

4.2 Data Preparation Pipeline

- **Collection:** Bare-act text is gathered for each statute in scope.
- **Cleaning:** Text is normalized — removing scanning artifacts, redundant headers/footers, and inconsistent formatting from source documents.
- **Chunking:** Each act is split into semantically coherent chunks (roughly section/sub-section level), since retrieval quality depends heavily on chunk boundaries — chunks that are too large dilute relevance, chunks that are too small lose legal context.
- **Embedding:** Each chunk is converted into a dense vector representation using an embedding model (OpenAI embeddings in the reference implementation), capturing its semantic meaning rather than just keywords.
- **Indexing:** Embeddings are stored in a vector database —
 - **ChromaDB** in the reference implementation — which supports fast approximate nearest-neighbour similarity search over the chunk vectors.

This pre-processing happens once (offline / at build time), producing a persistent vector store that the live application queries at runtime — the app itself never re-processes the raw legal text on every request.

5. Solution Architecture

The system is a Retrieval-Augmented Generation pipeline wrapped in a conversational session layer with caching, exposed through a chat UI. At a high level, a user's question is converted into a search query, matched against the pre-indexed legal text, and the matched passages are handed to an LLM which composes the final answer — the LLM never answers purely from its own memory.

5.1 High-Level Pipeline

Stage	What Happens
1. Data Collection	Bare-act texts for all covered statutes are gathered as the raw corpus.
2. Data Analysis / Prep	Text is cleaned, split into chunks, and embedded into vectors; vectors are indexed in ChromaDB to build the persistent legal knowledge base.
3. Query Contextualization	The user's latest message plus prior chat history is passed to the LLM to rewrite it into a standalone question (so pronouns/follow-ups like “what about the punishment for that?” resolve correctly).
4. RAG Retrieval	The standalone question is embedded and matched against the vector store using similarity search (top-k, with a minimum relevance/score threshold) to fetch the most relevant statutory chunks.
5. LLM Generation	Retrieved chunks + system prompt + chat history + the question are “stuffed” into a single prompt sent to the LLM (GPT-4o-mini in the reference build), which generates an answer grounded in that context.
6. Response Handling	The answer is cached (Redis) against a cache key derived from the query + session, and the session's chat history is updated with both the user question and the AI answer.
7. UI Delivery	The answer is streamed/rendered back into the chat interface, styled as an assistant message, continuing the conversation thread.

5.2 Detailed Flow: Data Collection → Analysis → RAG → LLM → Response → UI

Step 1 — Data Collection

Raw statutory text for each act is sourced and assembled into a corpus. This is a one-time (or periodically refreshed) offline step, independent of the live chat application.

Step 2 — Data Analysis & Preprocessing

The raw corpus is cleaned and split into chunks sized for good retrieval precision. Each chunk is passed through an embedding model to produce a fixed-length vector, and all vectors are written into a Chroma vector store on disk. This is effectively the “knowledge base build” stage — the output is a queryable, persisted vector index.

Step 3 — RAG (Retrieval)

At query time, the system does not do a plain similarity search on the raw user message. It first runs a history-aware retriever: the LLM reformulates the user's question into a standalone form using the chat history, then that reformulated question is embedded and matched against the vector store using similarity search with a score threshold (only chunks above a minimum relevance score are kept, and up to a fixed number of top matches are retrieved).

Step 4 — LLM (Generation)

The retrieved chunks are combined (“stuffed”) into a single context block and passed to the LLM alongside a system prompt (defining tone/role/constraints), the chat history, and the reformulated question. The LLM is instructed to answer using this retrieved context, which keeps the response grounded in the actual statute rather than the model's general training knowledge.

Step 5 — Response Handling

The generated answer is:

- Checked against a cache first (if an identical query for the session was already answered, the cached answer is returned instantly, skipping the LLM call entirely)
- Written to the cache after generation, keyed by a hash of the query + session
- Appended to the session's persistent chat history (stored via Redis) so later questions in the same conversation have full context

Step 6 — UI

The final answer is rendered in a chat bubble in the web interface, with the running conversation displayed above the input box — giving the user a familiar, low-friction chat experience rather than a search-results page.

5.3 Why Caching and Session Memory Matter

- **Cost & latency:** Legal questions are often repeated across users (e.g. “what is the punishment for theft?”). Caching avoids paying for and waiting on a fresh LLM call every time.
- **Conversational continuity:** Persisting chat history per session lets the assistant correctly resolve follow-up questions (“and what if it's a repeat offence?”) without the user having to restate context.

6. Technology Stack

Layer	Technology
Orchestration	LangChain — chains for history-aware retrieval and “stuffed-document” question answering
LLM	OpenAI GPT-4o-mini (pluggable — the architecture supports swapping in other LLM providers)
Embeddings	OpenAI Embeddings
Vector Store	ChromaDB (persisted on disk)
Caching / Session Store	Redis — caches LLM responses and stores per-session chat history
Backend Language	Python
Web / UI Framework	Streamlit (reference implementation); Django is also part of the broader stack
Multi-agent (roadmap)	CrewAI — planned for a future multi-agent architecture

7. User Interface

The interface is a single-page chat application:

- A title bar with the LawGlance logo and name
- A sidebar with a short description of the tool, a link to the project website, and a disclaimer that responses may not be 100% accurate (pilot-phase framing)
- A persistent, per-browser-session chat thread — each session gets a unique ID, and its history is reloaded from Redis on refresh
- A chat input box pinned to the bottom of the screen (“Have a legal question? Let's work through it.”)
- User and assistant messages rendered as distinct, styled chat bubbles
- The assistant's answer is revealed with a word-by-word typing animation for a more natural, conversational feel

This keeps the experience close to a familiar messaging app rather than a traditional legal-research tool, which is deliberate — the target user is a layperson, not a lawyer.

8. Our Contribution / Adaptation for This Project

For the major project, this architecture is studied as a reference and re-implemented with our own scope and extensions, which may include:

- Re-building the pipeline end-to-end (data collection → chunking → embedding → vector store → RAG → LLM → UI) as an original implementation rather than a fork
- A modern web UI (React / Next.js) in place of the Streamlit reference UI
- Evaluating GraphRAG (knowledge-graph-augmented retrieval) as an enhancement over plain vector similarity search, to better capture relationships between sections, definitions, and cross-references within the statutes
- Expanding or narrowing the legal domain coverage based on project scope and timeline
- Adding source citation of the exact act/section in each answer for transparency

9. Future Scope

- Multi-agent framework (specialized agents per legal domain) for more comprehensive answers
- Expansion to additional jurisdictions beyond India
- Voice-based interaction
- Multi-lingual legal assistance
- Personalized search and legal document drafting assistance
- Case management features (deadlines, appointments, tracked matters)

10. Limitations & Disclaimers

- The system is a legal information aid, not a substitute for professional legal advice — it does not represent clients or file cases.
- Answer quality is bounded by the completeness and currency of the indexed statutory text; it will not know about laws or amendments outside its knowledge base.
- As with any LLM-based system, residual risk of an inaccurate or incomplete answer remains, especially for nuanced or fact-specific legal scenarios — hence the in-app disclaimer shown to users.

Reference: github.com/lawglance/lawglance (Apache-2.0). This document explains the architecture as a basis for our own major-project implementation.